Software Rescue Mission

COMP 3415 | Assignment 2 | Santhosh Allada

8 November 2026. This report explains the original system, testing strategy, five diagnoses, three refactorings, express delivery change, and engineering judgments in Tasks 1-6.

Repository: https://github.com/salladaa/COMP3415-A2-Campus-Supply-Desk
Submission branch: main (the repository default). Application and test version described: 70aadbe1b8bf33ee7b6e54fab99e2582f6615a50. Later documentation and experiment commits do not change that application or its test suite.
Original starter: 82ed75e0714e50c086a13ee491f51b349814d8a4, preserved as assignment-starter. The checkout was clean, with one starter commit and no pre-existing local edits.

1. System understanding

Campus Supply Desk is a Python terminal application for viewing campus products, obtaining quotes, placing orders, viewing history, retrieving an order, and cancelling it. Users can choose an interactive menu or individual commands. Quotes calculate prices without reserving inventory or writing a save file. Placement validates a request, deducts stock, assigns an order ID, and saves an order snapshot. Monetary values are integer Canadian cents; the menu displays dollars.

The original supplydesk/cli.py defines parse_item(), build_parser(), and main() (lines 11-67). It converts PRODUCT_ID:QUANTITY text into a request, applies regular/pickup defaults, dispatches service calls, and prints JSON. With no command, main() launches the menu. supplydesk/menu.py defines read_request(), show_order(), and run_menu() (original lines 15-89); these collect input, display prices, ask for confirmation, and recover from input errors.

OrderService in supplydesk/service.py (original lines 6-148) owns validation, pricing, stock changes, placement, history, lookup, and cancellation. JsonStore in supplydesk/storage.py (lines 7-29) loads JSON state or initializes it from catalog.json and writes it after a mutation. tests/test_smoke.py contains the four supplied tests.

Execution flow: python -m supplydesk quote --item notebook:2 enters cli.main(). parse_item() produces a notebook quantity of two; parser defaults select regular/pickup. OrderService.quote() loads state, validates the request and stock, creates priced item lines, and returns a quote. The CLI prints total_cents = 1000. test_cli_defaults_and_invalid_exit and test_quote_has_no_side_effects reproduce this behavior and verify that a fresh quote creates no save file. In the final version, quote delegates validation/pricing to _prepare_quote(), still without saving.

The hardest original method is place_order() (lines 63-121): validation, pricing, order construction, inventory mutation, and saving are interleaved. Comparing it with quote() requires reading duplicated rules before locating the point where state changes. This supports the submitted checkpoint's central maintainability observation.

Source references throughout distinguish the original tag from the final application commit. Requirements are the assignment handout, FUNCTIONAL_SPEC.md, and the official CR-01 copied unchanged into docs/CR-01.md.

2. Testing strategy and baseline

All tests run in isolated temporary directories. No ordinary saved orders are modified. The runtime was CPython 3.12.14; the host's default Python 3.9.6 was not used because the README requires 3.10+. The application needs no third-party runtime packages.

Command at every stage: python -m unittest discover -s tests -v. The exact Python executable is recorded in each captured test log. evidence/run_tests.py records the actual executable, revision, working changes, output, and exit code. Starter results are in evidence/01-starter-tests.txt.

Four required categories

Normal: test_normal_placement_and_history places two notebooks with regular/pickup. Expected and actual: ORD-0001, status placed, total 1000 cents, stock 18, and the same order after reopening. This protects inventory and durable order creation.

Boundary: test_standard_delivery_boundary checks subtotals 9500 and 10000 cents. Expected and actual standard fees are 600 and 0, with totals 10100 and 10000. A student subtotal of 10000 receives a 1000-cent discount and still incurs 600 delivery, confirming that the free threshold is applied after discount. These checks matter because using the wrong comparison or applying the threshold before discount can charge or waive delivery incorrectly. Other tests cover last stock, quantity 99/100, student eligibility, and rounding.

Invalid: test_invalid_first_order_does_not_consume_id rejects zero quantity with ValueError. Expected and actual: no save file, stock 20, empty history, and the next valid order is ORD-0001 with 19 notebooks. test_invalid_requests_preserve_existing_state also checks malformed requests, unsupported options, and invalid quantity types against an existing save file, asserting its bytes remain unchanged. Rejected input must not lose inventory, add an order, or consume an order ID.

Behavior to preserve: test_quote_and_order_agree compares every quote field with the subsequent student delivery order. Independent expected and actual amounts are subtotal 8000, discount 800, delivery 600, total 7800. This matters because extracting shared pricing must not change a quoted amount when the order is placed. Independent amounts detect matching but incorrect calculations. Menu and CLI tests also protect confirmation, errors, and defaults.

Recorded results

Starter: 4 tests, all pass, exit 0 (01-starter-tests.txt).

Before refactoring: 23 tests, 5 failed assertions across 3 test methods, exit 1 (02-before-refactoring.txt).

After refactoring: the same 23 tests and the same 5 failures, exit 1 (03-after-refactoring.txt).

After separate defect fixes: 23 tests, all pass, exit 0 (04-after-defect-fixes.txt).

After CR-01 and at final verification: 32 tests, all pass, exit 0 (05-after-cr01.txt and 06-final-verification.txt).

The five baseline failures are two quote/placement failures for the exact student threshold, two for combined duplicate-line stock, and one for repeated cancellation. They are real failures, not skips or expected-failure annotations. Assertions were not weakened to hide them. Test counts refer to methods; several methods execute additional parameterized subcases.

3. Five code problem diagnoses

These are exactly the five diagnoses selected from Checkpoint A: three code smells, one testing issue, and one behavioral defect. Locations on this page and the next refer to assignment-starter (82ed75e). The two additional defects discovered by testing are discussed separately, not counted as extra Task 3 diagnoses.

Diagnosis 1 - Duplicated validation and pricing

Location: supplydesk/service.py, OrderService.quote(), lines 15-60, and place_order(), lines 65-112.

Problem and evidence: both methods repeat customer/fulfillment checks, item validation, catalog lookup, quantity and stock checks, line construction, the student discount, and the delivery calculation.

Consequence: changing a rule in only one copy can make the quoted amount disagree with the placed order. A maintainer must compare both blocks, increasing the review burden and the chance of divergent fixes.

Proposed improvement: extract a shared request-validation/pricing helper that receives the current state. Retain inventory changes, ID allocation, and saving in place_order(). Both public operations should load current state independently so an earlier quote cannot bypass stock revalidation.

Diagnosis 2 - Magic numbers in pricing

Location: supplydesk/service.py, quote() lines 47-52 and place_order() lines 97-102.

Problem and evidence: 5000, 10000, and 600 encode the student threshold, free-standard-delivery threshold, and standard fee without naming their purpose or units. The discount arithmetic also embeds percentage and rounding constants.

Consequence: readers must infer whether a number is a price, threshold, percentage, or rounding offset. This readability problem remains even if the duplicated block is consolidated, so it is distinct from Diagnosis 1.

Proposed improvement: introduce descriptive constants such as STUDENT_DISCOUNT_THRESHOLD_CENTS and STANDARD_DELIVERY_FEE_CENTS. Preserve values, comparison operators, and integer arithmetic during the refactoring; correct any rule mismatch separately.

Diagnosis 3 - Long menu method with mixed responsibilities

Location: supplydesk/menu.py, run_menu(), lines 45-89, especially quote/place lines 57-67 and cancellation lines 76-81.

Problem and evidence: one function prints the menu, dispatches every option, collects order input, previews prices, confirms placement/cancellation, displays results, and handles errors and exit signals.

Consequence: tracing a multi-step placement flow requires reading through unrelated options. Changes to confirmation handling are harder to review in a dispatcher containing all interaction details.

Proposed improvement: extract the longer quote/place and cancellation interactions. Keep short branches in run_menu() and preserve the outer exception handler so errors still return to the menu. Preserve prompts, y-confirmation behavior, and declined-action outcomes.

Evidence basis: these diagnoses reference the actual original methods, not generic examples. They motivate the three focused refactorings in Task 4; no replacement application or architecture framework was introduced.

3. Diagnoses continued and checkpoint review

Diagnosis 4 - Missing invalid-state preservation tests

Location: tests/test_smoke.py, OrderSmokeTests, lines 17-41.

Problem and evidence: the four starter tests check two successful quotes, order reopening, and one cancellation. None asserts that rejected placement preserves saved stock, history, and the next order number.

Consequence: an invalid-request regression could modify state while all four starter tests remain green. Passing the starter suite therefore provides insufficient evidence for safe refactoring.

Proposed improvement: add invalid-first-order and invalid-existing-state tests. Assert ValueError, unchanged stored data, and the next valid ID; use temporary files to isolate each test. The final suite implements both checks and tests malformed save files without silently resetting data.

Diagnosis 5 - Repeated cancellation inflates stock

Location: supplydesk/service.py, OrderService.cancel_order(), lines 133-148.

Problem and evidence: after locating an order, the method always restores each item quantity before setting status to cancelled. It does not first check whether the order is already cancelled. A two-notebook order changes stock 20 -> 18; cancellation restores 20; a second cancellation produces 22. test_repeated_cancellation_is_idempotent reproduced the final 22 != 20 failure.

Consequence: repeated cancellation creates inventory that does not exist, violating FUNCTIONAL_SPEC.md's Cancellation rule and making later stock decisions unreliable.

Proposed improvement: return the existing cancelled order before stock restoration or saving. Verify that repeat cancellation leaves saved bytes unchanged and retains the original items and totals. This is a behavioral defect fix, not a refactoring.

Corrections and additions to Checkpoint A

The submitted overview, notebook example, and all five diagnoses are supported. Some original references were imprecise: cli.py ends at line 67, not 66; run_menu() occupies lines 45-89 rather than the broad menu range given in the draft. This report replaces loose labels with explicit files, methods, and original/final revision context. The starter tests still consist of four successful-path checks.

The original checkpoint examples are now supported by reproducible automated tests. The four-category initial strategy was expanded into 19 new baseline test methods before application edits. A quote/order equality check was supplemented by independent expected amounts to avoid two implementations agreeing on the same wrong result.

Two additional specification defects emerged: student eligibility used subtotal > 5000 instead of >= 5000, and stock checks ignored combined quantities on duplicate product lines. They are recorded in docs/DEFECTS.md as testing discoveries. Neither changes the count of five Task 3 diagnoses. Together with repeated cancellation, they were corrected only after the refactoring checkpoint.

My original Checkpoint A remains the initial analysis. This report records the corrections and verification that followed it.

4. Three refactorings and verification

Refactoring commit: 54648e5. Baseline tests commit: 8c763c8. The selected smells are Diagnoses 1-3. The refactoring stage deliberately preserves known incorrect behavior so its effect can be assessed separately from defect repair.

Shared request preparation

Before: quote() and place_order() duplicated validation and price construction. Change: extract OrderService._prepare_quote(state, customer_type, items, fulfillment); quote loads and delegates, while placement loads, delegates, then adds ID/status, deducts stock, and saves.

Why appropriate: the duplicated operation is validation and quote construction; state mutation remains specific to placement. Passing the already-loaded state avoids an extra load inside the helper and keeps each operation's calculations consistent with its stock update.

After: one location controls validation and pricing. Final references: service.py lines 22-79 and 81-96. Verification: test_quote_and_order_agree, test_quote_has_no_side_effects, test_placement_rechecks_stock_after_quote, and invalid-request tests retain their outcomes after extraction.

Named monetary and arithmetic constants

Before: prices, thresholds, percentages, and rounding offsets appeared as unexplained literals. Change: name the student threshold/percentage, percentage denominator/half denominator, standard free threshold, and fee. The refactoring preserved the original strict greater-than comparison and integer half-up calculation.

Why appropriate: names explain purpose and units without changing the pricing algorithm. After: pricing rules can be read without guessing the meaning of 5000, 10000, and 600. Final references: service.py lines 6-12 and 63-70; the express constant was added later under CR-01.

Verification: standard-delivery boundary, below/above student threshold, half-cent rounding, and quote/order agreement tests preserve their baseline results. The exact-threshold failures remain visible until the separate fix.

Focused menu helpers

Before: run_menu() contained multi-step interactions alongside dispatch. Change: extract handle_quote_or_place() and handle_cancellation(); retain small catalog, history, and lookup branches inline and preserve the outer exception handling.

Why appropriate: extraction follows coherent user operations without creating one trivial function per branch. After: the dispatcher is easier to scan, and confirmation flows can be read independently. Final references: menu.py lines 45-63 and 66-97.

Verification: test_menu_defaults_decline_and_confirmation protects default values, decline behavior, placement, and cancellation. test_menu_errors_and_exit protects invalid input, retry behavior, EOF, and Ctrl+C.

Separate corrections and limits of evidence

The refactored suite reports exactly the baseline's five failed assertions and no new failures. This supports preservation for the tested cases; it is not a proof over every possible input. Commit a9ddcaf then corrects the three documented defects: >= at the student threshold, accumulated quantities per product, and an already-cancelled guard. The unchanged 23-test suite passes afterward. Those behavior changes are explicitly excluded from the refactoring claims. CR-01 begins from this corrected, refactored version.

5. Client change request: express delivery

The official request is preserved in docs/CR-01.md. Express must work through service calls, commands, and the menu; it always costs 1200 cents. Student discounts apply to merchandise only. Pickup, standard delivery, defaults, validation, stock, history, saved snapshots, and cancellation rules remain intact.

Prediction before implementation

Commit 4b6a209 records docs/CR01-PREDICTION.md before express code or tests. Predicted edits: service.py's shared helper and a new fee constant; cli.py's parser choices/help; menu.py's fulfillment prompt; README.md and FUNCTIONAL_SPEC.md; and new acceptance tests. No storage-schema or additional cancellation edits were expected. Planned checks covered all interfaces, pricing, lifecycle snapshots, invalid input, and existing behavior.

Implementation and acceptance evidence

Commit 70aadbe adds EXPRESS_DELIVERY_FEE_CENTS = 1200 and an express branch in _prepare_quote(). It extends CLI choices/help and the menu prompt. Discount arithmetic is unchanged from the separately corrected pre-CR version. Existing JSON fields already retain fulfillment and price snapshots.

1. Interfaces: test_official_examples_quote_and_place checks the service; test_cli_express_lifecycle_and_help runs real python -m supplydesk processes; test_menu_quote_place_retrieve_cancel scripts the actual terminal menu.
2. Fixed fee: test_express_fee_never_waived checks standard versus express, regular and student customers, and high-value orders. The official totals are reproduced: 2200, 8400, and 13200 cents. Standard delivery for three regular hoodies remains 12000.
3. Student rules: test_discount_threshold_and_rounding_exclude_delivery checks below/at/above 5000 and a controlled 5005-cent subtotal. The latter discounts 501 cents and totals 5704 with express. Adding delivery to 4500 of merchandise does not trigger a discount.
4. Existing delivery: the original smoke suite and test_standard_delivery_boundary still pass, including the post-discount standard threshold. Pickup remains free.
5. Lifecycle: test_lifecycle_snapshot_history_and_repeat_cancel verifies retrieval, reload, catalog changes, history, original monetary fields, repeated cancellation, restored stock, and continued IDs.
6. Validation/state: test_invalid_express_requests_preserve_state covers invalid customers, items, quantities, fulfillment values, unknown orders, and combined stock. test_quote_does_not_reserve_and_place_revalidates protects fresh stock checking.
7. Discovery/defaults: CLI help contains express and 1200; the menu displays the option and CAD fee. Menu/CLI default tests remain green. README section 7 and the functional specification describe the option.

Review against prediction

All predicted application files changed; storage and cancellation needed no express-specific edits. Shared pricing reduced the change to one calculation path for both quotes and orders. Named constants made units explicit. Menu extraction improved readability but had little direct effect on this change: only read_request() needed editing. Full acceptance mapping is in docs/CR01-REVIEW.md. All 32 tests pass; no addresses, timing guarantees, external services, or payments were added.

6. AI engineering judgment

I evaluated the following three recommendations using the code and test evidence. My responses and the relevant interactions are preserved in docs/AI-REVIEW.md. The third example was evaluated during final review, after CR-01.

Recommendation 1 - ACCEPT

AI recommendation: share request validation and pricing between quote and placement, using the state loaded for each operation.

My decision: Accept. Quotes and placed orders should apply the same rules. The same five assertions failed before and after refactoring, while the existing successful checks still passed.

Evidence/action: _prepare_quote() centralizes the logic; test_quote_and_order_agree also checks independent expected totals, while placement rechecks stock. The baseline and refactored logs show the same five failures. Implemented in 54648e5. Matching failure counts alone are not sufficient evidence, so successful regression assertions and the diff also support the decision.

Recommendation 2 - MODIFY

AI recommendation: extract every menu branch into a separate function. A narrower alternative was explicitly offered.

My decision: Modify. I chose to extract only the longer quote, placement, and cancellation flows because those branches handle multiple steps. The simpler menu options are easy to follow in place.

Evidence/action: only handle_quote_or_place() and handle_cancellation() were introduced. Quote and placement share one interaction helper. The menu confirmation, default, error, and exit tests preserve their results; short branches remain readable in run_menu(). Implemented in 54648e5.

Recommendation 3 - REJECT

AI candidate: simplify quantity validation from type(quantity) is not int to not isinstance(quantity, int). This candidate was evaluated in a temporary copy during final review, not applied to the application.

My decision: Reject. In Python, bool is a subclass of int, so isinstance(True, int) is True. The proposed check accepted quantity=True as one notebook, although the specification explicitly forbids boolean quantities. I kept the exact-type check because it enforces that rule.

Evidence/action: evidence/07-quantity-candidate.txt records the modified-copy run and focused probe: True was accepted and priced at 500 cents. The invalid-request tests failed, including quote and placement checks. The accepted invalid placement saved an additional order; later assertions then detected altered saved bytes and the wrong next ID. These cascading failures are consequences of one validation defect, not separate defects. The temporary copy was discarded and the real application retained its original check. evidence/evaluate_quantity_check.py reproduces the experiment. The final application suite still passes all 32 tests.

Repository and reproducibility

Repository: https://github.com/salladaa/COMP3415-A2-Campus-Supply-Desk

The completed work is on main, the default branch, so an ordinary clone contains the final application, tests, and report. Application/test commit: 70aadbe1b8bf33ee7b6e54fab99e2582f6615a50. Later commits add documentation and the isolated review experiment without changing those files. Original: assignment-starter / 82ed75e. Stages: tests 8c763c8; refactor 54648e5; fixes a9ddcaf; prediction 4b6a209; CR-01 70aadbe.

With Python 3.10+, run python -m unittest discover -s tests -v from the repository root. Expected: 32 tests, OK. Final revision evidence is evidence/08-submission-verification.txt; earlier verification remains in log 06. Interactive use: python -m supplydesk. All tests use temporary state. Concurrency, crash-safe saving, arbitrary manual save-file edits, payments, and shipping integrations remain outside the supplied scope.